CSE - 3212
Compiler Project Proposal

HALAL COMPILER

Submitted By:

Name: **Raihan Arefin**

Roll: **2107065**

Year: 3rd

Semester: 2nd

Submitted To:

Md. Badiuzzaman Shuvo

Lecturer

Department of Computer Science and Engineering

Khulna University of Engineering and Technology

Subah Nawar

Lecturer

Department of Computer Science and Engineering

Khulna University of Engineering and Technology

Date of Submission: 12 January,2026

Introduction

Halal Compiler is a small C-like programming language that replaces standard C keywords with Arabic transliterations (words/particles commonly seen in Arabic). The compiler transpiles the source language into standard C and then uses an existing C toolchain (gcc/clang) to produce an executable.

C uses English keywords and a strict syntax. This project explores how a new surface syntax can be mapped onto a standard backend language (C) without changing the underlying computation model.

Design Choice (Scope Control)

- Approach: a C-subset language with mostly C operators and punctuation, but custom keywords.
- Blocks: use { and } like C (simpler parsing; familiar to C learners).
- Compilation: halalc (My compiler) → generates out.c → compile via gcc/clang.

Keyword Mapping Table

Category	Halal keyword	C keyword	Notes
Control flow	iza	if	when/if
Control flow	wailla	else	otherwise
Control flow	iza_ila	else if	emitted as 'else if'
Control flow	ma_dama	while	as long as
Control flow	likulli	for	for each
Control flow	qif	break	stop
Control flow	wasil	continue	continue
Functions	irji	return	return
Types	adad	int	number
Types	harf	char	letter
Types	haqiqi	double	real
Types	fadi	void	empty

I/O	qul	printf	say
I/O	iqra	scanf	read

Example Program

Halal source (example):

```
adad main() {
  adad age = 20;

  iza (age > 18) {
    qul("kbeer!\n");
  } wailla {
    qul("sghayyar!\n");
  }

  irji 0;
}
```

Generated C output (conceptual):

```
#include <stdio.h>

int main() {
  int age = 20;

   if (age > 18) {
    printf("kbeer!\n");
  } else {
    printf("sghayyar!\n");
  }

  return 0;
}
```

Technical Approach

1. Lexical analysis (lexer): Converting input text to tokens.
2. Parsing: Building an AST for the chosen C subset.
3. Semantic analysis (lightweight): Symbol tables and basic checks.
4. Code generation: Emiting valid C.
5. Build/run: Invoking gcc/clang on generated C.